

Session 2: Control Flow

DIGITAL AND PUBLIC POLICY STREAM

Python Tutorial
Paul Jeu



Session outline

Decisions

if, elif and else
Conditions and
indentation

Repetition

for loops
range(start, stop, step)

Loop control

break and continue
else on loops

Hands-on: filter values and search a list

if statements

A condition decides whether an indented block runs.

A colon ends the if line.

Use four spaces for indentation. The next unindented line runs after the block.

Python 3

```
score = 72
if score >= 50:
    print("Pass")
print("Checked")
```

Output

```
Pass
Checked
```

elif and else

Python tests conditions in order.

Only the first matching branch runs.

else handles the remaining cases. It is optional.

Python 3

```
score = 72
if score >= 80:
    print("High")
elif score >= 50:
    print("Pass")
else:
    print("Retry")
```

Output

Pass

Conditions

Comparisons: ==, !=, <, <=, >, >=

and requires both conditions.

or requires at least one.

not reverses a truth value.

= assigns a value.

== compares values.

Python 3

```
age = 20
registered = True
if age >= 18 and registered:
    print("Eligible")
```

Output

```
Eligible
```

for statements

A for loop visits each item in order.

The loop variable takes the next value on each iteration.

Iterate directly over the items when you do not need their indices.

Python 3

```
cities = ["Paris", "Rome"]
for city in cities:
    print(city)
print("Finished")
```

Output

```
Paris
Rome
Finished
```

Filtering with for and if

Combine repetition with a decision.

Start with an empty result list, then append matching values.

Build a new list instead of removing items from the list you are visiting.

Python 3

```
scores = [42, 76, 50, 31]
passed = []
for score in scores:
    if score >= 50:
        passed.append(score)
print(passed)
```

Output

```
[76, 50]
```

The range() function

range(stop) begins at 0.

range(start, stop) begins at start.

The stop value is always excluded.

A range represents a sequence without building a list of all its values.

Python 3

```
print(list(range(4)))  
print(list(range(2, 6)))
```

Output

```
[0, 1, 2, 3]  
[2, 3, 4, 5]
```

Step size and direction

`range(start, stop, step)` changes by step each time.

A negative step counts down.

The default step is 1.

A zero step raises `ValueError`.

The wrong direction gives an empty range.

Python 3

```
print(list(range(1, 8, 2)))  
print(list(range(6, 0, -2)))  
print(list(range(6, 0)))
```

Output

```
[1, 3, 5, 7]  
[6, 4, 2]  
[]
```

Quick check: predict the result

Work individually, then compare with a neighbour.

1. Which values does n take?
2. What is the final total?
3. What changes if stop is 9?

Python 3

```
total = 0
for n in range(2, 8, 2):
    total = total + n
print(total)
```

break: stop a loop

break exits the nearest enclosing loop immediately.

Statements after the loop still run.

Useful when a search has found its first match.

Python 3

```
for score in [42, 76, 50]:  
    if score >= 50:  
        print(score)  
        break  
print("Done")
```

Output

```
76  
Done
```

continue: skip an iteration

continue skips the rest of the current iteration.

The next iteration still runs.

Useful for skipping values that should not be processed.

Python 3

```
for score in [42, 76, 50]:  
    if score < 50:  
        continue  
    print(score)
```

Output

```
76  
50
```

break and continue compared

break

```
for n in range(5):  
    if n == 2:  
        break  
    print(n)
```

Output: 0, then 1
The loop ends at n = 2.

continue

```
for n in range(5):  
    if n == 2:  
        continue  
    print(n)
```

Output: 0, 1, 3, then 4
Only the n = 2 print is skipped.

else on a loop

A loop else runs after normal completion.

A break skips it.

Align else with for.

Here, every item is checked and no match is found.

Python 3

```
for n in [3, 5, 7]:  
    if n % 2 == 0:  
        print("Found", n)  
        break  
else:  
    print("No even number")
```

Output

No even number

When does loop else run?

Normal completion

All items visited: else runs.

Empty iterable: else runs too.

continue does not prevent else.

Early exit

break skips else.

An exception that exits the loop also skips else.

For while, else runs when the condition becomes false.

Nested loops and break

break exits only the innermost loop that contains it.

The outer loop continues with its next item.

Trace the indentation to identify which loop a statement belongs to.

Python 3

```
for city in ["Paris", "Rome"]:  
    for n in range(3):  
        if n == 1:  
            break  
        print(city, n)
```

Output

```
Paris 0  
Rome 0
```

Hands-on Session: Control Flow

Hands-on: find the first matching value

Use this list

```
values = [0, 5, 8, 12, 7]
```

Write a loop that:

1. Skips zero using continue.
2. Finds the first positive multiple of 4.
3. Prints that value and stops using break.
4. Prints "No match" with loop else if none exists.

Hands-on discussion and solution

Why skip zero first?

Zero is divisible by 4, but the task asks for a positive value.

Why does 12 never print?

The first match triggers break.

Try [0, 5, 7] and []. Both produce No match.

Python 3

```
values = [0, 5, 8, 12, 7]
for n in values:
    if n == 0:
        continue
    if n > 0 and n % 4 == 0:
        print(n)
        break
else:
    print("No match")
```

Output

8

Recap and readings

if selects a branch.

for visits items in order.

range excludes its stop value.

break ends a loop.

continue skips to the next iteration.

Loop else runs after normal completion.

[Python documentation: More Control Flow Tools](#)

Sections 4.1–4.5